

05 SQL Modifications and SQL for Data Analytics

7.3.2 SQL INSERT Statement

Purpose

- Used to insert data into relational databases.

Basic Syntax

```
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, value2, value3);
```

Examples

1. Inserting a single tuple

```
INSERT INTO PRODUCT VALUES ('980', 'Chateau Angelus, Grand Cru Cla
ssé, 1960', 'red', 6);
```

2. Explicit column specification (Recommended)

```
INSERT INTO PRODUCT (PRODNR, PRODNAME, PRODTYPE, AVAILABLE_
QUANTITY)
VALUES ('980', 'Chateau Angelus, Grand Cru Classé, 1960', 'red', 6);
```

3. Inserting multiple records

```
INSERT INTO PRODUCT (PRODNR, PRODNAME, PRODTYPE, AVAILABLE_
QUANTITY)
VALUES ('980', 'Chateau Angelus', 'red', 6),
```

```
('1000', 'Domaine de la Vougeraie', 'white', 2),  
( '1002', 'Leeuwin Estate Cabernet Sauvignon', 'white', 20);
```

4. Insert partial data

```
insert into user(username) values ('Donna');
```

5. Insert Combined with Select

```
INSERT INTO EMAIL  
SELECT email FROM USER WHERE EMAIL IS NOT NULL;
```

Using INSERT with a Subquery

- Inserts supplier numbers of inactive suppliers into a new table.

```
INSERT INTO INACTIVE_SUPPLIERS (SUPNR)  
SELECT SUPNR FROM SUPPLIER  
EXCEPT  
SELECT SUPNR FROM SUPPLIES;
```

- Important to **respect constraints** (e.g., `NOT NULL`, foreign key integrity) to prevent errors.

7.3.3 SQL DELETE Statement

Purpose

- Removes records from a table.

Basic Syntax

```
DELETE FROM table_name WHERE condition;
```

Examples

1. Deleting a specific product

```
DELETE FROM PRODUCT WHERE PRODNR = '1000';
```

2. Removing suppliers with NULL status

```
DELETE FROM SUPPLIER WHERE SUPSTATUS IS NULL;
```

3. Using a Subquery to Delete Chardonnay Wines

```
DELETE FROM SUPPLIES  
WHERE PRODNR IN (SELECT PRODNR FROM PRODUCT WHERE PRODNAME LIKE '%CHARD%');
```

4. Correlated Subquery Example

- Removes suppliers with no corresponding records in **SUPPLIES**:

```
DELETE FROM SUPPLIER R  
WHERE NOT EXISTS (  
    SELECT PRODNR FROM SUPPLIES S  
    WHERE R.SUPNR = S.SUPNR  
);
```

5. Self-referencing DELETE

- Deletes products priced more than twice their average price:

```
DELETE FROM SUPPLIES S1  
WHERE S1.PURCHASE_PRICE >  
(SELECT 2 * AVG(S2.PURCHASE_PRICE) FROM SUPPLIES S2 WHERE S1.PRODNR = S2.PRODNR);
```

Caution

- **Cascading deletes** may be triggered depending on `ON DELETE CASCADE` constraints.
 - `DELETE FROM PRODUCT` without a `WHERE` clause removes all records.
-

7.3.4 SQL UPDATE Statement

Purpose

- Modifies existing data in a table.

Basic Syntax

```
UPDATE table_name  
SET column1 = value1, column2 = value2  
WHERE condition;
```

Examples

1. Updating the quantity of a product

```
UPDATE PRODUCT  
SET AVAILABLE_QUANTITY = 26  
WHERE PRODNR = '0185';
```

2. Setting a column to its default value

```
UPDATE SUPPLIER SET SUPSTATUS = DEFAULT;
```

3. Using a Subquery to Update Records

- Extends delivery time for a specific supplier:

```
UPDATE SUPPLIES  
SET DELIV_PERIOD = DELIV_PERIOD + 7
```

```
WHERE SUPNR IN (SELECT SUPNR FROM SUPPLIER WHERE SUPNAME =  
'Deliwines');
```

4. Using a Correlated Subquery

- Ensures supplier 68 offers minimum prices:

```
UPDATE SUPPLIES S1  
SET (PURCHASE_PRICE, DELIV_PERIOD) =  
(SELECT MIN(PURCHASE_PRICE), MIN(DELIV_PERIOD)  
FROM SUPPLIES S2 WHERE S1.PRODNR = S2.PRODNR)  
WHERE SUPNR = '68';
```

5. Combining UPDATE with ALTER TABLE

- Adds a new column and updates it with conditional logic.

```
ALTER TABLE SUPPLIER ADD SUPCATEGORY VARCHAR(10) DEFAULT 'SIL  
VER';  
  
UPDATE SUPPLIER  
SET SUPCATEGORY = CASE  
    WHEN SUPSTATUS BETWEEN 70 AND 90 THEN 'GOLD'  
    WHEN SUPSTATUS > 90 THEN 'PLATINUM'  
    ELSE 'SILVER'  
END;
```

Caution

- Referential integrity constraints (e.g., `ON UPDATE CASCADE`) determine how updates propagate to related tables.

7.4 SQL Views

Purpose

- **A virtual table** based on a query.
- Simplifies complex queries, enhances security, and ensures logical data independence.

Creating a View

```
CREATE VIEW view_name AS  
SELECT column1, column2 FROM table_name WHERE condition;
```

Examples

1. Defining a View

```
CREATE VIEW TOPSUPPLIERS AS  
SELECT SUPNR, SUPNAME FROM SUPPLIER WHERE SUPSTATUS > 50;
```

2. Defining a Dependent View

```
CREATE VIEW TOPSUPPLIERS_SF AS  
SELECT * FROM TOPSUPPLIERS WHERE SUPCITY = 'San Francisco';
```

3. Defining an Aggregated View

```
CREATE VIEW ORDEROVERVIEW(PRODNR, PRODNAME, TOTQUANTITY)  
AS  
SELECT P.PRODNR, P.PRODNAME, SUM(POL.QUANTITY)  
FROM PRODUCT P LEFT JOIN PO_LINE POL ON P.PRODNR = POL.PRODNR  
GROUP BY P.PRODNR;
```

Using a View

- Querying a view works like a table:

```
SELECT * FROM TOPSUPPLIERS_SF;  
SELECT * FROM ORDEROVERVIEW WHERE PRODNAME LIKE '%CHAR  
D%';
```

Query Modification

- The database modifies queries to refer to base tables automatically:**Becomes**

```
SELECT * FROM TOPSUPPLIERS WHERE SUPCITY = 'Chicago';
```

```
SELECT SUPNR, SUPNAME FROM SUPPLIER WHERE SUPSTATUS > 50 A  
ND SUPCITY = 'Chicago';
```

View Materialization

- Some databases store views physically for performance (Materialized Views).

Updatable Views

- Some views allow `INSERT`, `UPDATE`, and `DELETE`, but must meet strict conditions:
 - No `DISTINCT`
 - No aggregate functions (`SUM`, `AVG`, etc.)
 - No subqueries in `WHERE`
 - No `GROUP BY`, `UNION`, `INTERSECT`, `EXCEPT`

WITH CHECK OPTION

- Prevents updates that violate the view's condition.

```
CREATE VIEW TOPSUPPLIERS AS  
SELECT SUPNR, SUPNAME FROM SUPPLIER WHERE SUPSTATUS > 50  
WITH CHECK OPTION;
```

- **Example:**

```
UPDATE TOPSUPPLIERS SET STATUS = 20 WHERE SUPNR = '32';
```

- This **fails** because `STATUS = 20` does not satisfy `SUPSTATUS > 50`.